Lycée César Baggio Term STI2D SIN **Travaux pratiques**

Le lampadaire autonome — du capteur de lumière à la machine à états, sur ESP32



Allumé à fond quand quelqu'un passe, en veille le reste de la nuit : c'est vous qui l'écrivez.

Durée : 3 h 00 min **Compétences :** CO5.8-SIN2 · CO7.3-SIN1 · CO6.5-SIN1 · CO7.2 · CO3.3

Nom : Prénom : Classe : Date : **/ 20**

0. En-tête pédagogique

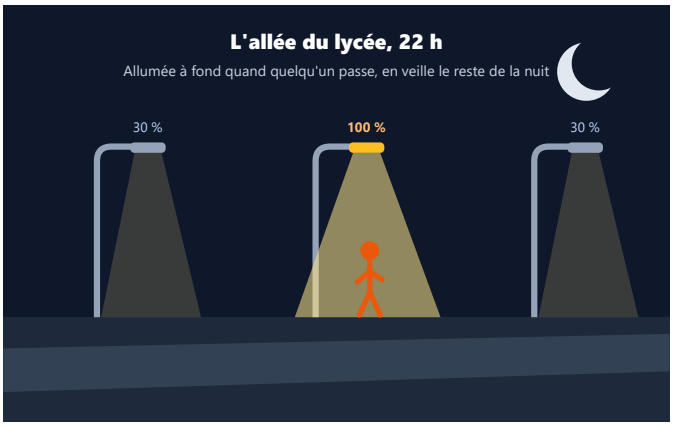
Rubrique	Contenu
Classe	Terminale STI2D — Spécialité 2I2D, enseignement spécifique SIN (TSTI2D2)
Durée	3 h, en binôme
Prérequis	T1 : quantum, code N, binaire et hexadécimal ; Python de base (variables, `if`, `while`) ; hystérésis (TP « cuve d'eau de pluie »)
Matériel	Un poste par binôme, Chrome, Wokwi (sans compte) ; fichiers `diagram_lampadaire.json` et `lampadaire.py`
Documents ressources	DR1 : documentation du capteur de lumière · DR2 : diagramme d'états du lampadaire · DR3 : mémo MicroPython

Code	Compétence	Questions
CO7.3-SIN1	Expérimenter des moyens matériels d'acquisition, de traitement, de stockage et de restitution de l'information pour aider à la conception d'une chaîne d'information.	Q1 à Q4, Q7
CO5.8-SIN2	Rechercher et écrire l'algorithme de fonctionnement puis programmer la réponse logicielle relative au traitement d'une problématique posée.	Q5, Q6, Q9
CO3.3	Identifier et caractériser le fonctionnement temporel d'un produit ou d'un processus.	Q8
CO7.2	Mettre en œuvre un scénario de validation, interpréter les résultats et qualifier le produit.	Q10
CO6.5-SIN1	Interpréter les résultats d'une simulation et conclure sur la performance de la solution.	Q11

Connaissances associées : 2.4.2c Conversion analogique/numérique · 2.4.3a Encodage de l'information · 2.4.3b Algorithmique · 3.4.2b Diagramme d'états · 4.3.5b Codage dans un langage spécifique · 4.3.5c Mise au point · 3.4.3e Système temps-réel.

1. Mise en situation

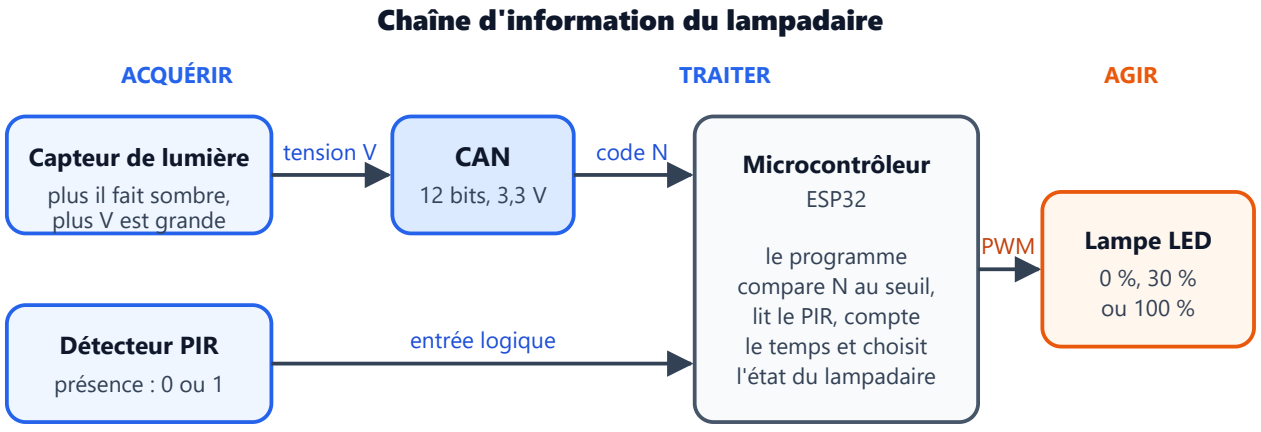
L'allée du lycée, 22 h



L'allée qui mène au lycée compte **12 lampadaires LED de 60 W**, allumés à fond de 17 h 30 à 7 h 30 en hiver, soit environ **10 kWh par nuit** pour une allée presque toujours vide. La commune veut un éclairage **autonome** : éteint le jour, en **veille à 30 %** la nuit, à **100 %** pendant 60 s dès qu'un piéton est détecté. Le bureau d'études vous confie le prototype du programme, sur un ESP32 simulé.

Problématique

Comment programmer un lampadaire qui décide seul de son niveau d'éclairage, sans clignoter et sans jamais rater un piéton ?



$q = PE / 2^n$ et N = partie entière de V / q

Figure 1 : *Chaîne d'information du lampadaire*

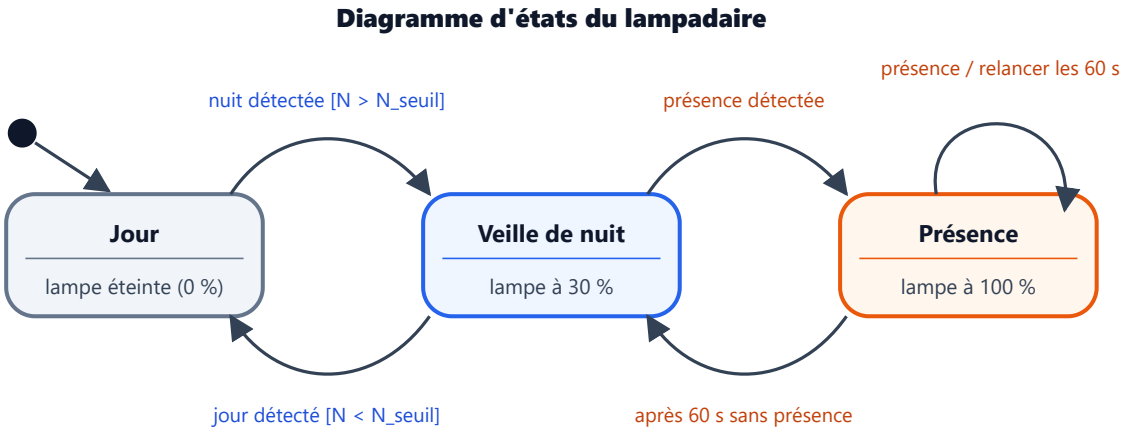
DR1 — Le capteur de lumière (module à photorésistance)

- Photorésistance en série avec une résistance de $10\ \text{k}\Omega$, alimentée en $3{,}3\ \text{V}$; la sortie **AO** est reliée à l'entrée analogique 34 de l'ESP32.
- **Plus il fait sombre, plus la tension AO est grande.**
- Points de fonctionnement :

Éclairement	Situation	Tension AO
15 lux	crépuscule clair	2,61 V
10 lux	tombée de la nuit (seuil d'éclairage public retenu)	2,75 V
7 lux	nuit tombée	2,86 V

- Dans Wokwi, pendant la simulation, **cliquez sur le capteur** pour régler l'éclairement (en lux) avec un curseur.

DR2 — Diagramme d'états du lampadaire (cahier des charges)



Une transition s'écrit : événement [condition] / action.

DR3 — Mémo MicroPython (ESP32)

Instruction	Effet
`n = capteur.read()`	code du CAN, de 0 à 4095 (12 bits, 0 à 3,3 V)
`pir.value()`	1 si une présence est détectée, 0 sinon
`regler_lampe(30)`	lampe à 30 % (fonction fournie, PWM)
`t = time.ticks_ms()`	temps écoulé depuis le démarrage, en ms
`time.ticks_diff(t2, t1)`	durée entre deux instants, en ms
`etat = "VEILLE"`	une variable texte peut mémoriser l'état courant

Dans Wokwi, **cliquez sur le détecteur PIR** pendant la simulation, puis sur « Simulate Motion » : sa sortie passe à 1 pendant 5 s.

2. Travail demandé

Pour démarrer (10 min) : ouvrez `wokwi.com/projects/new/micropython-esp32`. Dans l'onglet `diagram.json`, collez tout le contenu du fichier `diagram_lampadaire.json` ; dans l'onglet `main.py`, collez

`lampadaire.py`. Lancez la simulation (▶) : le moniteur série affiche `N = …` toutes les 0,2 s.

Partie A — Mesurer la lumière (reprise du CAN, 50 min)

Question 1 (1 pt)

Calculer le quantum q du CAN de l'ESP32, en mV.

Question 2 (2 pts)

Relever le code N affiché pour un éclairement de 1 000 lux, 100 lux, 10 lux et 1 lux (curseur du capteur). Dans quel sens N varie-t-il quand la nuit tombe ? Est-ce cohérent avec le DR1 ?

Question 3 (2 pts)

Compléter la ligne `v = 0` du programme pour calculer la tension v à partir du code n. Vérifier à 10 lux que la tension affichée correspond au DR1.

Question 4 (2 pts)

Calculer le seuil N_{seuil} correspondant à la tombée de la nuit (DR1), l'écrire en hexadécimal, puis le renseigner dans le programme (`N_SEUIL`).

Partie B — Le jour et la nuit (35 min)

Question 5 (2 pts)

Programmer à la place du `TODO Q5` : la lampe est à 30 % s'il fait nuit, éteinte sinon. Tester en faisant varier l'éclairement autour de 10 lux.

Question 6 (2 pts)

Un soir nuageux, l'éclairement oscille entre 9 et 11 lux ; les phares d'une voiture ajoutent quelques lux. **Expliquer** ce que fait la lampe avec le programme de la Q5, puis **proposer** deux seuils (hystérésis) à partir du DR1 : passage en nuit à 7 lux, retour au jour à 15 lux. Donner `N_NUIT` et `N_JOUR`.

Partie C — Détecter les piétons, compter le temps (60 min)

Question 7 (1 pt)

Observer : lancer « Simulate Motion » sur le PIR et relever, avec le moniteur série, combien de temps `presence` reste à 1.

Question 8 (1 pt)

Un camarade propose, quand un piéton est détecté : `regler_lampe(100)` puis `time.sleep(60)`. **Expliquer** pourquoi cette solution ne respecte pas le DR2.

Question 9 (4 pts)

Programmer le diagramme d'états du DR2, avec l'hystérésis de la Q6 et une temporisation non bloquante : une variable `etat` (`"JOUR"`, `"VEILLE"` ou `"PRESENCE"`), une variable `debut_presence`, puis, à chaque tour de boucle, les transitions et le réglage de la lampe. Afficher `etat` dans le `print`.

Partie D — Valider et conclure (25 min)

Question 10 (2 pts)

Rédiger puis réaliser un protocole de test qui vérifie **chaque transition** du DR2 : pour chaque essai, l'action réalisée dans Wokwi, l'état attendu et l'état observé.

Question 11 (1 pt)

Sur une nuit d'hiver de 14 h, on compte 1 h cumulée de présence. **Calculer** l'énergie consommée par les 12 lampadaires toujours à fond, puis avec le programme, et **conclure** sur la problématique.

3. Bilan

À retenir

- Le code fourni par un CAN se convertit en tension par $V \approx N \times q$; un seuil se calcule en ..... pour être comparé directement à la valeur lue.
- Un seuil unique fait ..... la commande autour du seuil ; deux seuils (une .....) suppriment ce défaut.
- Un diagramme d'états se programme avec une variable qui mémorise l'..... courant et des tests qui codent les ..... .
- Une temporisation ..... mémorise un instant (`ticks_ms`) et compare à chaque tour : le programme continue de lire ses capteurs.
- Un protocole de test vérifie ..... du cahier des charges.

4. Barème

Partie	Questions	Points
A — Mesurer la lumière (CAN)	Q1 à Q4	7
B — Jour, nuit, hystérésis	Q5, Q6	4
C — Présence et temporisation	Q7 à Q9	6
D — Validation et bilan énergétique	Q10, Q11	3
Total		**20**